Processor Organization

Processor does:

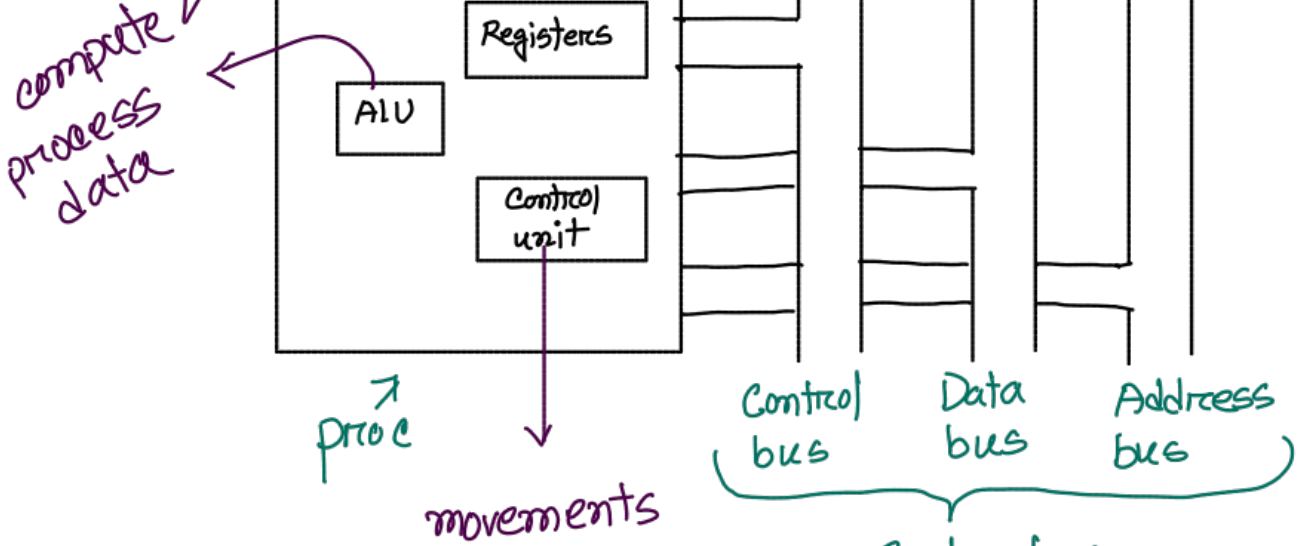
1. Fetch instr: Read 'em from Mem (reg, cache, M.M)
2. Interpret instr: Decode → Determine action.
3. Fetch Data: read from memory / I/O module.
4. Process Data: arithmetic / logical operation.
5. Write Data: write results to memory or I/O module.

→ To do all these:

- Proc must store some data temporarily
- remember loc of last instr.

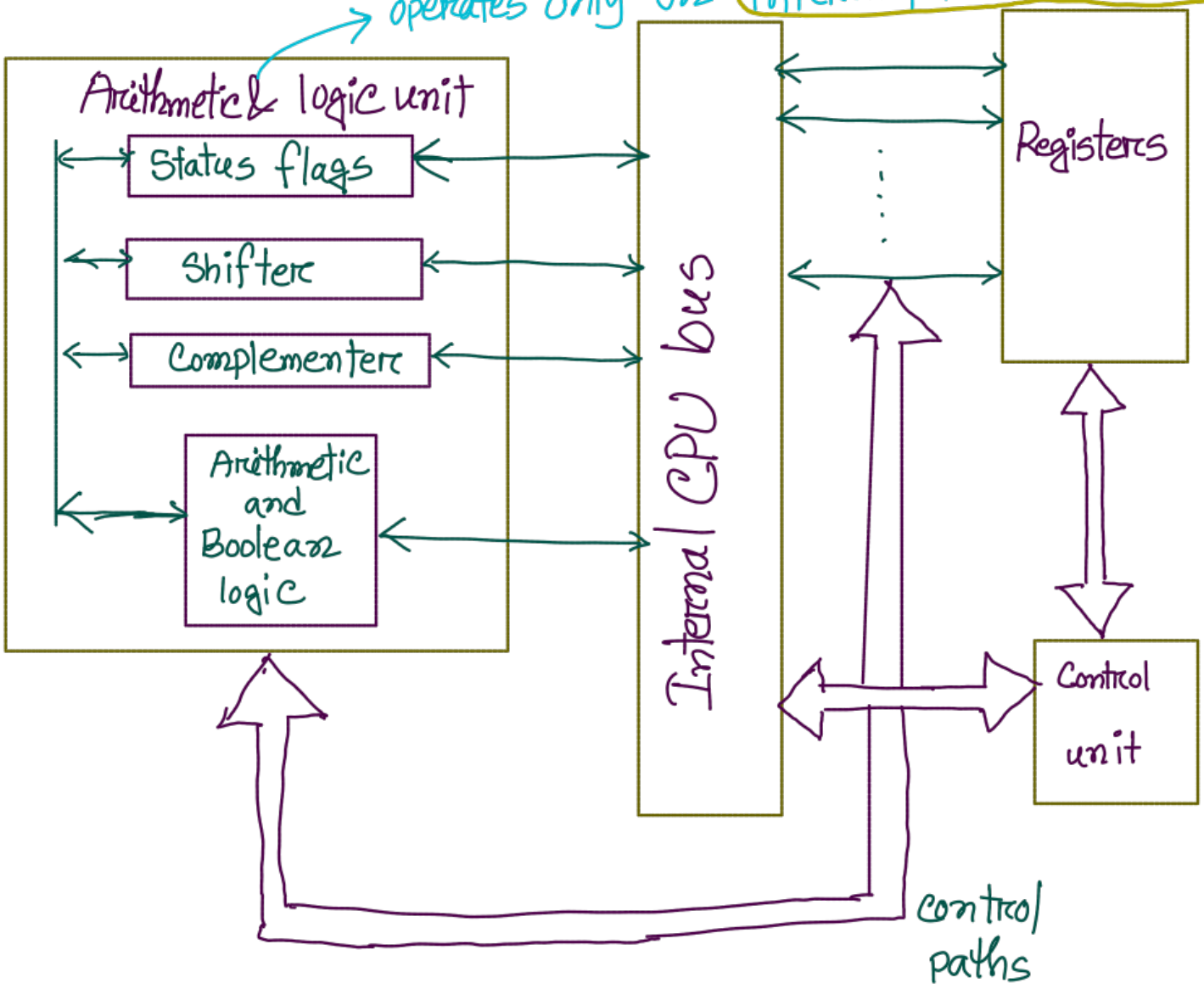
Proc needs its own mem.





of instr & data into
& out of the processor
+ controls ALU.

operates only on internal proc memory



Register Organization

(within) In proc, there is a set of registers that are above the M.M & cache, ↓ 2 roles.

1. User-visible reg: minimize M.M reference by optimizing use of reg. for ML and AL programmer.

2. Control & Status reg:
   - ↓ CU uses to control proc operation
   - privileged OS progs use .. to control prog execution

User visible reg

↳ can be referenced thru ML that the proc executes.

1. General-purpose:
   ↳ assign many functions

→ use in instr set is orthogonal to the operation.

[any reg can contain the operand for any op-code). → true general purpose usage.

limitation: dedicated registers for floating-point & stack operation.

Can be used for addressing functions

↳ register indirect, displacement.

2. Data reg : only holds data, can't calc operand address.

3. Address reg : somewhat general purpose / devoted to particular address mode.

↳ **Segment Pointer:** holds base addresses in segments in segmented mem sys.

Index Reg: idx addressing (sometimes

Stack Pointer: Points to the top of the stack for stack based operation.

Design Considerations:

General-purpose vs. Specialized Registers: Specialized registers save instruction bits but reduce flexibility.

Number of Registers: Typically optimal between 8–32; more registers don't significantly reduce memory references, but some modern RISC systems use hundreds.

Register Length: Must be long enough for addresses; some systems allow two registers to store double-length values.

Condition Codes:

Set by arithmetic and data operations to indicate results (e.g., zero, negative, overflow).

Used in conditional branching but add complexity to hardware and software.

Some systems avoid condition codes by evaluating conditions directly in branch instructions

Adv	Disadv
Condⁿ codes set by →	

Advantages vs. Disadvantages of Condition Codes:

Reduce the need for explicit COMPARE/TEST instructions.

Simplify conditional branching.

Enable multiway branching.

Allow easy saving/restoring during subroutine calls.

However, they add hardware complexity, require special synchronization in pipelined systems, and may still require additional non-condition-code instructions.

Subroutine calls may save registers automatically (by the processor) or require programmers to manage saving/ restoring

1. Because condition codes are set by normal arithmetic and data movement instructions, they should reduce the number of COMPARE and TEST instructions needed.	1. Condition codes add complexity, both to the hardware and software. Condition code bits are often modified in different ways by different instructions, making life more difficult for both the micro programmer and compiler writer.
2. Conditional instructions, such as BRANCH, are simplified relative to composite instructions, such as TEST and BRANCH.	2. Condition codes are irregular; they are typically not part of the main data path, so they require extra hardware connections.
3. Condition codes facilitate multiway branches. For example, a TEST instruction can be followed by two branches, one on less than or equal to zero and one on greater than zero.	3. Often, condition code machines must add special non-condition-code instructions for special situations anyway, such as bit checking, loop control and atomic semaphore operations.
4. Condition codes can be saved on the stack during subroutine calls, along with other register information.	4. In a pipelined implementation, condition codes require special synchronization to avoid conflicts.

In some machines, a subroutine call will result in all the user-visible registers being saved so that they can be restored on return. The processor performs the saving and restoring as part of the execution of `CALL` and `RETURN` instructions. This allows each subroutine to use the user-visible registers independently. Other machines put the responsibility on the programmer, so they need to include instructions to do this in the program.

Enable multiway branching.

Allow easy saving/restoring during subroutine calls.

However, they add hardware complexity, require special

Control & Status register

PC: addrs of next instr. to fetch

IR: stores current instr. being executed. (holds fetched instr.)

MAR: contains addrs in memory for data access.

MBR: Holds data being transferred to/from memory.

ALU processes data from MBR & user-visible registers.

Program Status Word (PSW)

↳ contains status flags

- Sign, Zero, Carry, Equal, Overflow
- Interrupt Enable/Disable, Supervisor Mode.

Additional Regs

1. Interrupt Vector Register: Manages vectored interrupts.
2. System Stack Pointer: Handles stack operations.

3. Page Table Pointer: Supports virtual memory.

4. I/O Control Registers: Manage I/O operations.

Design Considerations:

OS Support: Registers designed to support OS functions and privileges.

Memory vs. Register Allocation: Balance between speed (registers) and cost (memory).

Summary of Microprocessor Register Organization

Two different register organizations are compared:

1. **First Organization:**
   - **Registers:** 8 data registers, 9 address registers (all 32-bit).
   - **Data Registers:** Used for data manipulation and indexing.
   - **Address Registers:** Store 32-bit addresses; two serve as stack pointers (user & OS modes).
   - **Additional Registers:** A 32-bit program counter and a 16-bit status register.
   - **Design Philosophy:** Regular instruction set, no special-purpose registers, optimized for code efficiency by partitioning registers.

2. **Second Organization:**
   - **Registers:** All are special-purpose, though some have general-purpose capabilities.

- **Data Registers:** 4x 16-bit, usable in byte or word mode, some used implicitly (e.g., accumulator for multiplication).
- **Pointer & Index Registers:** 4x 16-bit, used implicitly for addressing.
- **Segment Registers:** 4x 16-bit, three dedicated for instructions, data, and stack segmentation.
- **Additional Registers:** Instruction pointer and status/control flags.
- **Design Philosophy:** Compact encoding via implicit register usage, at the cost of flexibility.

Key Takeaways
- No universally optimal register organization exists—design choices depend on trade-offs between flexibility, efficiency, and backward compatibility.
- Some microprocessors extend older architectures, embedding previous register structures to maintain compatibility.

This describes a microprocessor with a **special-purpose register organization**, meaning each register has a specific role rather than being fully general-purpose. Here's a breakdown:

Key Points:
1. **Data Registers (4 x 16-bit)**
   - These can store 16-bit or byte-sized values.
   - Some instructions allow them to be used as general-purpose registers.
   - However, certain operations **require specific registers** (e.g., multiplication always uses the accumulator).

2. **Pointer and Index Registers (4 x 16-bit)**
   - Used for addressing memory locations.
   - Often used **implicitly** by the processor (e.g., without explicitly specifying them in the instruction).
   - Each contains a **segment offset**, meaning they help in addressing different memory segments.

3. **Segment Registers (4 x 16-bit)**
   - **Three of them are dedicated to:**
     - **Current instruction segment** (used for branching).
     - **Data segment** (used for accessing variables).
     - **Stack segment** (used for function calls and local variables).
   - These segmentation rules help with **memory organization but reduce flexibility** (because each segment register has a fixed role).

4. **Additional Registers**
   - **Instruction Pointer:** Keeps track of the next instruction to execute.
   - **Status & Control Flags:** 1-bit flags indicating conditions like zero, carry, overflow, etc.

Main Idea
- This approach **compacts** instruction encoding because many registers are used implicitly.
- However, it **reduces flexibility** because you cannot freely assign registers for any task—each has a fixed role.
- This design is typical of older x86 processors, where segmentation was a key memory management technique.

The registers are a **mix of data and address registers**:

1. **Data Registers (4 x 16-bit)** → Used to store and manipulate data.
   - Can hold **8-bit or 16-bit values**.
   - Sometimes used as general-purpose registers.
   - Some instructions require specific data registers (e.g., multiplication always uses the accumulator).

2. **Pointer & Index Registers (4 x 16-bit)** → Used for addressing memory (so they are address registers).
   - Used **implicitly** in memory-related operations.
   - Store **segment offsets**, helping access different parts of memory.

3. **Segment Registers (4 x 16-bit)** → Also address registers.
   - Define **memory segments** (Instruction, Data, Stack).
   - Ensure compact instruction encoding by avoiding explicit memory references.

Conclusion
- **Data Registers** → Store and manipulate values.
- **Pointer & Index Registers + Segment Registers** → Used for addressing memory.
- Some **data registers** can also act as address registers** in certain cases, but **not all registers are purely data or purely address registers**—some serve mixed roles.

B### **Instruction Pipelining Explained**

What is Instruction Pipelining?
Instruction pipelining is a technique used in modern processors to improve performance by overlapping the execution of multiple instructions. Instead of executing one instruction at a time from start to finish, pipelining divides the execution process into multiple stages, where different instructions are processed at different stages simultaneously—similar to an **assembly line** in a manufacturing plant.

Pipelining Strategy: The Assembly Line Analogy
Think of a factory that manufactures products in different steps:
1. **Step 1:** Raw materials are gathered.
2. **Step 2:** The parts are assembled.
3. **Step 3:** The product is painted.
4. **Step 4:** The product is packaged and shipped.

Instead of waiting for one product to finish before starting the next, the factory works on multiple products at the same time, each in a different step. This speeds up production.

A **processor pipeline** works the same way:
- The first instruction starts execution.
- While it is being executed, the second instruction is fetched.

- The third instruction is being decoded.
- The fourth instruction is calculating its operands, and so on.

By processing multiple instructions in different stages

simultaneously, the processor increases efficiency and instruction throughput.

Basic Two-Stage Pipelining

Before looking at a complex pipeline, let's consider a simple **two-stage pipeline**:

1. **Fetch Stage:** The processor fetches an instruction from memory and places it in a buffer.
2. **Execute Stage:** The processor executes the instruction while fetching the next one.

This process is called **instruction prefetch** or **fetch overlap** because fetching happens while execution is in progress. This improves speed but also requires **additional registers** to store intermediate results.

Limitations of the Two-Stage Pipeline

Although pipelining increases speed, it has some drawbacks:

1. **Execution Time Variability:**
   - Fetching an instruction takes **less time** than executing it.
   - Since execution involves reading/storing operands and performing calculations, it takes longer.
   - The fetch stage often has to **wait** for the execute stage to finish.

2. **Conditional Branching Problem:**
   - In cases of **branch instructions** (e.g., `if` conditions or loops), the processor does not immediately know the next

instruction's address.
  - The fetch stage might fetch the **wrong instruction**, leading to wasted cycles.
  - A possible solution: The processor can always **fetch the next instruction** assuming the branch is **not taken**.
    - If the branch is **not taken**, this instruction is useful.
    - If the branch **is taken**, the fetched instruction must be discarded, which is a **waste of cycles**.

Despite these challenges, **pipelining still significantly improves execution speed**.

Six-Stage Pipeline Breakdown

To further improve performance, processors introduce **more pipeline stages**. A six-stage pipeline typically consists of:

1. **Fetch Instruction (FI)**
  - The next instruction is read from memory and stored in a buffer.

2. **Decode Instruction (DI)**
  - The processor determines the operation code (**opcode**) and the operand specifiers (which registers/ memory locations are involved).

3. **Calculate Operands (CO)**
  - If the instruction needs memory access (e.g., `LOAD` or `STORE`), the processor calculates the **effective address** using techniques like:
    - Displacement addressing

- Register indirect addressing
- Indirect addressing

4. **Fetch Operands (FO)**
   - The required operands are loaded from memory.
   - If operands are stored in registers, no memory access is needed.

5. **Execute Instruction (EI)**
   - The operation (e.g., addition, subtraction, comparison) is performed.
   - The result is stored in a temporary register.

6. **Write Operand (WO)**
   - The final result is stored in memory or in a register, completing the instruction execution.

Advantages of a Six-Stage Pipeline
- **More instructions in execution at the same time.**
- **Better CPU utilization, reducing idle time.**
- **Faster instruction throughput.**
- **More efficient use of processor registers and memory.**

Potential Drawbacks
1. **Pipeline Stalls (Hazards)**
   - If an instruction depends on the result of a previous instruction that is still executing, the pipeline **stalls** (delays execution).
   - Example:
     ```assembly
     ADD R1, R2, R3  ; R1 = R2 + R3
     ```

```
SUB R4, R1, R5  ; R4 = R1 - R5 (R1 is not ready yet!)
```

- The second instruction has to **wait** for the first one to complete.
- This delay is called a **data hazard**.

2. **Control Hazards**
   - Occur due to **conditional branches** where the next instruction is unknown until the branch is resolved.
   - This can cause pipeline flushing (discarding partially executed instructions).

3. **Structural Hazards**
   - If different pipeline stages require the **same hardware resources**, conflicts can occur (e.g., two stages trying to access memory at the same time).

Conclusion

Instruction pipelining is an essential technique for increasing processor performance by allowing multiple instructions to be processed simultaneously in different stages. While it comes with challenges such as hazards and branch delays, modern CPUs include optimizations like **branch prediction** and **out-of-order execution** to minimize these issues. By increasing the number of pipeline stages, modern processors achieve **higher instruction throughput and better overall performance**.